



Proyecto - Entrega 05

Informe Testing Proyecto

Ingeniería de Software 1

Integrantes

Tomás Mozo González

Daniel Ferney Gomez Rojas

Julian David Nieto Rodriguez

Diego Fernando Tique Ramirez

Eduin Julian Barrera Rincon

Tabla

Funcionalidad	Pruebas a realizar
Publicación exitosa de tareas	Publicar una tarea de paseo de mascotas. Publicar una tarea de limpieza del hogar. Publicar una tarea con pago decimal. Publicar una tarea con un monto elevado.
Validación de datos obligatorios	Título vacío. Descripción vacía. Categoría vacía. Categoría con espacios. Ubicación vacía.
Validación de reglas de negocio	Pago igual a cero. Pago negativo. Pago NaN. Pago Infinity. Tipo de pago inválido. Usuario igual a cero. Usuario negativo.
Gestión de estados e historial de tareas	Actualizar el estado de una tarea exitosamente. Generar registro automático en el historial de estados. Impedir la modificación de una tarea en estado Finalizada Impedir el salto directo de estado Creada a Finalizada Retornar un arreglo vacío si la tarea consultada no posee historial

Pruebas Unitarias por Funcionalidad

Se implementaron pruebas unitarias para las principales funcionalidades de la aplicación. Las pruebas fueron desarrolladas por medio de la herramienta **Jest**, qué es un framework de pruebas que viene incluido por defecto en nuestro proyecto desarrollado con **NestJS** y **TypeScript**.

1.1 Publicación de tareas

Esta funcionalidad permite a los usuarios publicar nuevas tareas dentro de la plataforma, especificando información como título, descripción, categoría, ubicación, tipo de remuneración, monto ofrecido y usuario responsable de la publicación.

Casos de prueba implementados

Se implementaron pruebas unitarias utilizando Jest para verificar el correcto funcionamiento de la funcionalidad de publicación de tareas. Las pruebas se agruparon en tres compartimientos principales de la funcionalidad:

Publicación exitosa de tareas

Se verifica que el sistema permite crear correctamente una tarea cuando toda la información suministrada es válida.

Los escenarios evaluados fueron:

- Publicación de una tarea de paseo de mascotas.
- Publicación de una tarea de limpieza del hogar.
- Publicación de una tarea con remuneración decimal.
- Publicación de una tarea con remuneración de valor elevado.

En todos los casos se verificó que:

- La tarea fue creada correctamente.
- Los espacios adicionales en los campos de texto fueron eliminados antes del almacenamiento.
- El servicio de persistencia (**Prisma**) recibió la información esperada.

```
const cases = [
  {
    name: 'User publishes a pet walking task',
    dto: {
      title: ' Walk the dog ',
      description: ' One hour walk ',
      category: ' Pets ',
      location: ' Bogotá ',
      paymentType: 'HOUR',
      amount: 20,
      userId: 1,
    },
    shouldThrow: false,
  },
  {
    name: 'User publishes a house cleaning task',
    dto: {
      title: ' Clean the house ',
      description: ' Deep cleaning ',
      category: ' Home ',
      location: ' Medellín ',
      paymentType: 'DAY',
      amount: 150,
      userId: 2,
    },
    shouldThrow: false,
  },
  {
```

Validación de campos obligatorios

Se verificó que el sistema rechazará la creación de tareas cuando alguno de los campos obligatorios no contenía información válida.

Los escenarios evaluados fueron:

- Título vacío.
- Descripción vacía.
- Categoría vacía.
- Categoría compuesta únicamente por espacios.
- Ubicación vacía.

En cada uno de estos casos se comprobó que el sistema lanzara una excepción (`BadRequestException`) y que la tarea no fuera almacenada en la base de datos.

```
create(createTaskDto: CreateTaskDto) {  
  const validations = [  
    {  
      valid: createTaskDto.title.trim(),  
      message: 'Title is required',  
    },  
    {  
      valid: createTaskDto.description.trim(),  
      message: 'Description is required',  
    },  
    {  
      valid: createTaskDto.category.trim(),  
      message: 'Category is required',  
    },  
    {  
      valid: createTaskDto.location.trim(),  
      message: 'Location is required',  
    },  
    {  
      valid: Number.isFinite(createTaskDto.amount),  
      message: 'Invalid amount',  
    },  
    {  
      valid: createTaskDto.userId > 0,  
      message: 'Invalid user',  
    },  
    {  
      valid: createTaskDto.amount > 0,  
      message: 'Amount must be greater than zero',  
    },  
  ],  
}
```

Validación de reglas de negocio

Se verificó el cumplimiento de las restricciones definidas por la lógica del negocio para la publicación de tareas.

Los escenarios evaluados fueron:

- Remuneración igual a cero.
- Remuneración negativa.
- Valor de remuneración no numérico (NaN).
- Valor de remuneración infinito (Infinity).
- Tipo de remuneración inválido.
- Identificador de usuario igual a cero.
- Identificador de usuario negativo.

En todos estos escenarios se verificó que la operación fuera rechazada y que no se realizara ningún intento de almacenamiento en la base de datos.

```
{
  name: 'User enters amount equal to zero',
  dto: {
    title: 'Move furniture',
    description: 'Second floor',
    category: 'Moving',
    location: 'Cali',
    paymentType: 'DAY',
    amount: 0,
    userId: 1,
  },
  shouldThrow: true,
  message: 'Amount must be greater than zero',
},

{
  name: 'User enters negative amount',
  dto: {
    title: 'Paint wall',
    description: 'Bedroom',
    category: 'Home',
    location: 'Bogotá',
    paymentType: 'DAY',
    amount: -50,
    userId: 1,
  },
  shouldThrow: true,
  message: 'Amount must be greater than zero',
},
```

Casos límite evaluados

Las siguientes condiciones límite fueron evaluadas mediante pruebas unitarias:

- Publicación con monto igual a cero.
- Publicación con monto negativo.
- Publicación con valores especiales (NaN e Infinity).
- Publicación con tipo de remuneración no permitida.
- Publicación con campos obligatorios vacíos.
- Publicación con campos que contienen únicamente espacios en blanco.
- Publicación con identificadores de usuario inválidos.
- Publicación con remuneraciones decimales.
- Publicación con remuneraciones de gran magnitud.

```
{
  valid: createTaskDto.location.trim(),
  message: 'Location is required',
},
{
  valid: Number.isFinite(createTaskDto.amount),
  message: 'Invalid amount',
},
{
  valid: createTaskDto.userId > 0,
  message: 'Invalid user',
},
{
  valid: createTaskDto.amount > 0,
  message: 'Amount must be greater than zero',
},
{
  valid: ['HOUR', 'DAY'].includes(createTaskDto.paymentType),
  message: 'Invalid payment type',
},
},
];

const failedValidation = validations.find(v => !v.valid);

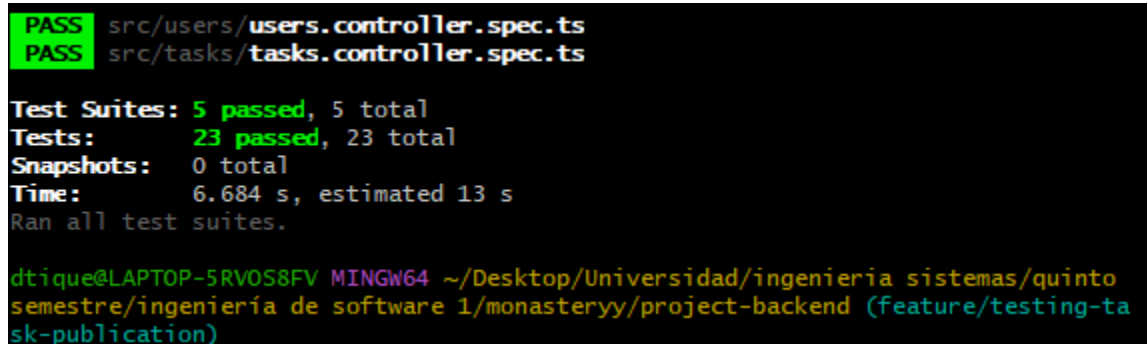
if (failedValidation) {
  throw new BadRequestException(failedValidation.message);
}
```

Ejecución de las pruebas

Las pruebas unitarias fueron ejecutadas mediante el siguiente comando:

```
npx jest src/tasks/tasks.service.spec.ts
```

El resultado obtenido fue satisfactorio, ejecutándose correctamente la totalidad de los escenarios de prueba implementados, sin presentarse fallos.



```
PASS src/users/users.controller.spec.ts
PASS src/tasks/tasks.controller.spec.ts

Test Suites: 5 passed, 5 total
Tests:       23 passed, 23 total
Snapshots:   0 total
Time:        6.684 s, estimated 13 s
Ran all test suites.

dtique@LAPTOP-5RV0S8FV MINGW64 ~/Desktop/Universidad/ingenieria sistemas/quinto
semestre/ingeniería de software 1/monasteryy/project-backend (feature/testing-ta
sk-publication)
```

Coverage

Para calcular la cobertura de las pruebas se utilizó el comando:

```
npx jest src/tasks/tasks.service.spec.ts --coverage
```

Los indicadores obtenidos permiten evaluar el porcentaje del código ejercitado durante la ejecución de las pruebas unitarias implementadas para la funcionalidad de publicación de tareas.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	57.14	65.9	29.03	55.55	
src	43.33	50	75	37.5	
...controller.ts	100	75	100	100	6
app.module.ts	0	100	100	0	1-13
app.service.ts	100	100	100	100	
main.ts	0	0	0	0	1-17
src/prisma	42.85	100	0	40	
prisma.module.ts	0	100	100	0	1-8
...ma.service.ts	66.66	100	0	57.14	11-19
src/tasks	61.9	63.63	30.76	60	
...controller.ts	72.22	64.28	16.66	68.75	12,21,30,35,40
tasks.module.ts	0	100	100	0	1-13
tasks.service.ts	76.47	62.5	42.85	71.42	70-97
src/tasks/dto	100	100	100	100	
...e-task.dto.ts	100	100	100	100	
...e-task.dto.ts	100	100	100	100	
...tasks/entities	0	100	100	0	
task.entity.ts	0	100	100	0	1
src/users	52.77	75	16.66	50	
...controller.ts	72.22	75	16.66	68.75	12,17,22,27,32
users.module.ts	0	100	100	0	1-11
users.service.ts	54.54	75	16.66	44.44	9-32
src/users/dto	100	100	100	100	
...e-user.dto.ts	100	100	100	100	
...e-user.dto.ts	100	100	100	100	
...users/entities	0	100	100	0	
user.entity.ts	0	100	100	0	1
tasks.service.ts	76.47	62.5	42.85	71.42	70-97
src/tasks/dto	100	100	100	100	

Los resultados obtenidos para el servicio evaluado fueron:

- Statements: 76.47%
- Branches: 62.50%
- Functions: 42.85%
- Lines: 71.42%

1.2 Búsqueda y filtrado de tareas

Esta funcionalidad le permite a los usuarios consultar tareas publicadas disponibles aplicando diferentes filtros, tales como categoría, ubicación y remuneración mínima.

Casos de prueba implementados

1.) Filtrados básicos (Categoría, ubicación y remuneración mínima)

Se verifica que el sistema construya correctamente cada uno (por separado) de los filtros, generando la consulta correspondiente,a partir del siguiente código:

```

it('should filter tasks by category', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll('Mascotas');

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: 'Mascotas',
      location: undefined,
      amount: undefined,
    },
  });
});

it('should filter tasks by location', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll(undefined, 'Bogotá');

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: undefined,
      location: 'Bogotá',
      amount: undefined,
    },
  });
});

it('should filter tasks by minimum amount', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll(undefined, undefined, 30);

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: undefined,
      location: undefined,
      amount: {
        gte: 30,
      },
    },
  });
});

```

2.) Casos Límites Evaluados

Se ejecutaron las siguientes pruebas unitarias para casos límite:

- Categoría inexistente: Retorna lista vacía .

- Ubicación inexistente Retorna lista vacía.
- Consulta sin filtros: Realiza consulta sin aplicar filtrado.
- Aplicación simultánea de múltiples filtros: Construye consulta con combinación de filtros.

```

5 describe('TasksService', () => {
79   it('should handle non-existing location filter', async () => {
80     jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);
81
82     const result = await service.findAll(undefined, 'Antartida');
83
84     expect(prismaService.task.findMany).toHaveBeenCalledWith({
85       where: {
86         category: undefined,
87         location: 'Antartida',
88         amount: undefined,
89       },
90     });
91
92     expect(result).toEqual([]);
93   });
94
95   it('should return empty result when category does not exist', async () => {
96     jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);
97
98     const result = await service.findAll('CategoriaInexistente');
99
100    expect(prismaService.task.findMany).toHaveBeenCalledWith({
101      where: {
102        category: 'CategoriaInexistente',
103        location: undefined,
104        amount: undefined,
105      },
106    });
107
108    expect(result).toEqual([]);
109  });
110
111
112
113   it('should return all tasks when no filters are provided', async () => {
114     jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);
115
116     await service.findAll();
117
118     expect(prismaService.task.findMany).toHaveBeenCalledWith({
119       where: {
120         category: undefined,
121         location: undefined,
122         amount: undefined,
123       },
124     });
125   });

```

```

it('should apply multiple filters simultaneously', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll(
    'Mascotas',
    'Bogotá',
    20,
  );

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: 'Mascotas',
      location: 'Bogotá',
      amount: {
        gte: 20,
      },
    },
  });
});

```

Las pruebas mediante **Jest** se ejecutaron utilizando el siguiente comando en la terminal:

npx jest src/tasks/tasks.service.spec.ts

El resultado de las pruebas es el siguiente:

```

C:\Users\tomas\OneDrive\Documentos\UNAL\project-backend>npx jest src/tasks/tasks.service.spec.ts
PASS src/tasks/tasks.service.spec.ts
  TasksService
    ✓ should be defined (9 ms)
    ✓ should filter tasks by category (3 ms)
    ✓ should filter tasks by location (1 ms)
    ✓ should filter tasks by minimum amount (1 ms)
    ✓ should handle non-existing location filter (2 ms)
    ✓ should return empty result when category does not exist (2 ms)
    ✓ should return all tasks when no filters are provided (1 ms)
    ✓ should apply multiple filters simultaneously (1 ms)
  Test Suites: 1 passed, 1 total
  Tests:      8 passed, 8 total
  Snapshots:  0 total
  Time:       1.03 s, estimated 2 s
  Ran all test suites matching src/tasks/tasks.service.spec.ts.

```

`npx jest src/tasks/tasks.service.spec.ts --coverage`

Para la generación de los indicadores de cobertura para la funcionalidad evaluada a partir de las pruebas unitarias realizadas. Como se puede ver a continuación:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	7.26	8.62	4.76	5.96	
src	0	0	0	0	
app.controller.ts	0	0	0	0	1-10
app.module.ts	0	100	100	0	1-14
app.service.ts	0	100	0	0	1-6
main.ts	0	0	0	0	1-17
src/applications	0	0	0	0	
applications.controller.ts	0	0	0	0	1-32
applications.module.ts	0	100	100	0	1-11
applications.service.ts	0	0	0	0	1-39
src/applications/dto	0	100	100	0	
create-application.dto.ts	0	100	100	0	1-10
update-application.dto.ts	0	100	100	0	1-4
src/applications/entities	0	100	100	0	
application.entity.ts	0	100	100	0	1
src/prisma	42.85	100	0	40	
prisma.module.ts	0	100	100	0	1-8
prisma.service.ts	66.66	100	0	57.14	11-19
src/tasks	19.44	25	16.66	16.66	
tasks.controller.ts	0	0	0	0	1-40
tasks.module.ts	0	100	100	0	1-13
tasks.service.ts	63.63	83.33	33.33	55.55	11, 35-48
src/tasks/dto	0	100	100	0	

Los resultados obtenidos para el servicio evaluado fueron:

- Statements: 63.63%
- Branches: 83.33%
- Functions: 33.33%
- Lines: 55.55%

1.3 Gestión de estados e historial de tareas

Esta funcionalidad permite administrar el ciclo de vida de las tareas en la plataforma (estados como 'Creada', 'En Progreso', 'Finalizada' y 'Cancelada'), controlando que las transiciones sean válidas mediante reglas de negocio y registrando automáticamente cada cambio en un historial global.

Casos de prueba implementados

Se diseñaron e implementaron pruebas unitarias utilizando Jest para validar tanto el flujo correcto de los estados como el control de errores ante acciones no permitidas:

- **Actualizar Estado exitoso:** Se verifica que, al pasar una tarea a un estado válido (por ejemplo, de 'Creada' a 'En Progreso'), el estado se actualice correctamente y se genere de forma automática el respectivo registro en el historial.

- **Validación de transiciones (Casos Límite):**
 - **Modificación de tareas finalizadas:** Se comprueba que el sistema bloquee cualquier intento de alterar o mover una tarea que ya se encuentra en estado 'Finalizada', lanzando la excepción correspondiente (*"No se puede modificar una tarea que ya está Finalizada"*).
 - **Salto de estado inválidos:** Se valida la regla de negocio que impide que una tarea salte directamente desde 'Creada' hasta 'Finalizada' sin haber pasado por un estado intermedio, arrojando un error controlado (*"No se puede saltar de Creada directamente a Finalizada"*).
- **Obtener Historial:** Se evalúa que la consulta filtre de manera correcta el historial por el identificador de la tarea. Como caso límite, se verifica que devuelva un arreglo vacío ([]) si la tarea consultada no registra cambios previos.

```

4 describe('Pruebas Unitarias - Gestión de Estados (Daniel)', () => {
5   let service: EstadoTareasService;
6   let tareaMock: Tarea;
7
8   beforeEach(() => {
9     service = new EstadoTareasService();
10
11     while(historialEstados.length > 0) {
12       historialEstados.pop();
13     }
14
15     tareaMock = {
16       id: 'task-123',
17       titulo: 'Hacer el testing',
18       estado: 'Creada'
19     };
20   });
21
22   // TEST FUNCIONALIDAD 1
23   describe('Funcionalidad 1: Actualizar Estado', () => {
24     test('Debe cambiar el estado exitosamente y registrar en el historial', () => {
25       const resultado = service.actualizarEstado(tareaMock, 'En Progreso');
26       expect(resultado.estado).toBe('En Progreso');
27       expect(historialEstados.length).toBe(1);
28     });
29   });
30
31   // TEST FUNCIONALIDAD 2
32   describe('Funcionalidad 2: Validar Transiciones (Casos Límite)', () => {
33     test('CASO LÍMITE: Debe lanzar error si se intenta cambiar una tarea Finalizada', () => {
34       tareaMock.estado = 'Finalizada';
35       expect(() => {
36         service.actualizarEstado(tareaMock, 'En Progreso');
37       }).toThrow('No se puede modificar una tarea que ya está Finalizada');
38     });
39
40     test('CASO LÍMITE: Debe impedir salto inválido de Creada a Finalizada', () => {
41       expect(() => {
42         service.actualizarEstado(tareaMock, 'Finalizada');
43       }).toThrow('No se puede saltar de Creada directamente a Finalizada');
44     });
45   });
46
47   // TEST FUNCIONALIDAD 3
48   describe('Funcionalidad 3: Obtener Historial', () => {
49     test('CASO LÍMITE: Debe retornar un arreglo vacío si la tarea no tiene historial', () => {
50       const historial = service.obtenerHistorialPorTarea('id-inexistente');
51       expect(historial).toEqual([]);
52     });
53   });
54 });

```

EVIDENCIA DE EJECUCIÓN

```

PS C:\Users\vasus\OneDrive\Desktop\project-backend> npm run test -- TestStateHomeworks
> jest TestStateHomeworks

PASS src/TestStateHomeworks.spec.ts
  Pruebas Unitarias - Gestión de Estados (Daniel)
    Funcionalidad 1: Actualizar Estado
      ✓ Debe cambiar el estado exitosamente y registrar en el historial (2 ms)
    Funcionalidad 2: Validar Transiciones (Casos Límite)
      ✓ CASO LÍMITE: Debe lanzar error si se intenta cambiar una tarea Finalizada (7 ms)
      ✓ CASO LÍMITE: Debe impedir salto inválido de Creada a Finalizada (1 ms)
    Funcionalidad 3: Obtener Historial
      ✓ CASO LÍMITE: Debe retornar un arreglo vacío si la tarea no tiene historial (2 ms)

Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:  0 total
Time:        0.64 s
Ran all test suites matching TestStateHomeworks.
PS C:\Users\vasus\OneDrive\Desktop\project-backend>

```

Coverage

Se utilizó el comando:

npx jest src/TestStateHomeworks.spec.ts --coverage

```
PS C:\Users\asus\OneDrive\Desktop\project-backend> npx jest src/TestStateHomeworks.spec.ts --coverage
PASS src/TestStateHomeworks.spec.ts
Pruebas Unitarias - Gestión de Estados (Daniel)
Funcionalidad 1: Actualizar Estado
  ✓ Debe cambiar el estado exitosamente y registrar en el historial (2 ms)
Funcionalidad 2: Validar Transiciones (Casos Límite)
  ✓ CASO LÍMITE: Debe lanzar error si se intenta cambiar una tarea Finalizada (9 ms)
  ✓ CASO LÍMITE: Debe impedir salto inválido de Creada a Finalizada (1 ms)
Funcionalidad 3: Obtener Historial
  ✓ CASO LÍMITE: Debe retornar un arreglo vacío si la tarea no tiene historial (2 ms)

-----
File                                     % Stmts   % Branch   % Funcs   % Lines   Uncovered Line #s
-----
All files                               9.86     15.38     8.82     11.62
src                                     31.25     50         37.5     36.58
  app.StateHomeworks.ts                 83.33     80         75     88.23    20,46
  app.controller.ts                     0         0         0         0     1-10
  app.module.ts                         0        100        100         0     1-13
  app.service.ts                       0        100         0         0     1-6
  main.ts                              0         0         0         0     1-17
src/prisma                             0        100         0         0
  prisma.module.ts                     0        100        100         0     1-8
  prisma.service.ts                   0        100         0         0     1-19
src/tasks                              0         0         0         0
  tasks.controller.ts                  0         0         0         0     1-40
  tasks.module.ts                     0        100        100         0     1-13
  tasks.service.ts                     0         0         0         0     1-48
src/tasks/dto                          0        100        100         0
  create-task.dto.ts                  0        100        100         0     1-30
  update-task.dto.ts                  0        100        100         0     1-4
src/tasks/entities                     0        100        100         0
  task.entity.ts                      0        100        100         0     1
src/users                              0         0         0         0
  users.controller.ts                  0         0         0         0     1-32
  users.module.ts                     0        100        100         0     1-11
  users.service.ts                     0         0         0         0     1-32
src/users/dto                          0        100        100         0
  create-user.dto.ts                  0        100        100         0     1
  update-user.dto.ts                  0        100        100         0     1-4
src/users/entities                     0        100        100         0
  user.entity.ts                      0        100        100         0     1
-----
Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        4.706 s
Ran all test suites matching src/TestStateHomeworks.spec.ts.
```

1.4 Verificación y seguridad de usuarios

Esta funcionalidad presenta las pruebas unitarias desarrolladas para los requerimientos RF_9, RF_10 y RF_16 del sistema. Las pruebas fueron implementadas utilizando Jest sobre NestJS y se enfocan en validar reglas de negocio relacionadas con seguridad, autenticación y protección de información sensible. También se presenta la evidencia de ejecución y el coverage obtenido.

RF_9 - Verificación de cuenta

Permite verificar si un usuario puede interactuar con las tareas dependiendo de la verificación de correo, teléfono y estado de bloqueo.

Casos de prueba

Caso	Resultado esperado
Email verificado	True

Teléfono verificado	True
Ambos Verificados	True
Ninguno verificado	False
Cuenta bloqueada	False

```
it('should validate account verification rules', () => {
  const cases = [
    {
      name: 'email verified',
      user: {
        emailVerified: true,
        phoneVerified: false,
        accountBlocked: false,
      },
      expected: true,
    },
    {
      name: 'phone verified',
      user: {
        emailVerified: false,
        phoneVerified: true,
        accountBlocked: false,
      },
      expected: true,
    },
    {
      name: 'both verified',
      user: {
        emailVerified: true,
        phoneVerified: true,
      },
      expected: true,
    },
  ],
```

```
{
  name: 'none verified',
  user: {
    emailVerified: false,
    phoneVerified: false,
  },
  expected: false,
},
{
  name: 'account blocked',
  user: {
    emailVerified: true,
    phoneVerified: true,
    accountBlocked: true,
  },
  expected: false,
},
],
```

RF_10 - Validación de contraseña

Valida que la contraseña tenga:

- mínimo 8 caracteres
- al menos una letra
- al menos un número
- al menos un símbolo

Casos de prueba

Caso	Resultado esperado
Contraseña válida	true
Menos de 8 caracteres	false

Sin números	false
Sin símbolos	false
Sin letras	false
Vacía	false

```
it('should validate passwords with multiple input cases', () => {
  const cases = [
    {
      name: 'valid password',
      password: 'Segura123!',
      expected: true,
    },
    {
      name: 'exactly 8 valid characters',
      password: 'Abc123!@',
      expected: true,
    },
    {
      name: 'less than 8 characters',
      password: 'Seg1!',
      expected: false,
    },
    {
      name: 'without numbers',
      password: 'Contrasena!',
      expected: false,
    },
    {
      name: 'without symbols',
      password: 'Contrasena123',
      expected: false,
    },
  ],
```

```
{
  name: 'without letters',
  password: '12345678!',
  expected: false,
},
{
  name: 'empty password',
  password: '',
  expected: false,
},
},
```

RF_16 - Restricción de información sensible

Ocultar la información de contacto mientras la tarea no haya sido aceptada.

Casos de prueba

Estado	Resultado esperado
ACCEPTED	Muestra información
accepted	Muestra información
PUBLISHED	Ocultar información

CANCELLED	Ocultar información
UNKNOWN	Ocultar información

```
it('should protect sensitive contact information', () => {
  const contactInfo = {
    contactNumber: '3001234567',
    email: 'usuario@test.com',
  };

  const cases = [
    {
      name: 'accepted task',
      status: 'ACCEPTED',
      expected: contactInfo,
    },
    {
      name: 'accepted task lowercase',
      status: 'accepted',
      expected: contactInfo,
    },
    {
      name: 'published task',
      status: 'PUBLISHED',
      expected: {
        contactNumber: 'Información restringida',
        email: 'Información restringida',
      },
    },
  ],
```

```
{
  name: 'cancelled task',
  status: 'CANCELLED',
  expected: {
    contactNumber: 'Información restringida',
    email: 'Información restringida',
  },
},
{
  name: 'unknown status',
  status: 'UNKNOWN',
  expected: {
    contactNumber: 'Información restringida',
    email: 'Información restringida',
  },
},
],
```

EVIDENCIA DE EJECUCIÓN

```
$ npm run test -- src/security/security.service.spec.ts

> freetime-backend@0.0.1 test
> jest src/security/security.service.spec.ts

PASS src/security/security.service.spec.ts
  SecurityService
    ✓ should be defined (7 ms)
    ✓ should validate passwords with multiple input cases (2 ms)
    ✓ should validate account verification rules (1 ms)
    ✓ should protect sensitive contact information (4 ms)

Test Suites: 1 passed, 1 total
Tests: 4 passed, 4 total
Snapshots: 0 total
Time: 0.635 s, estimated 1 s
Ran all test suites matching src/security/security.service.spec.ts.
```

COVERAGE

```
$ npm run test:cov -- src/security/security.service.spec.ts
```

File	% Stats	% Branch	% Funcs	% Lines	Uncovered line #s
All files	10.66	23.63	9.09	10.4	
src	0	0	0	0	
app.controller.ts	0	0	0	0	1-10
app.module.ts	0	100	100	0	1-13
app.service.ts	0	100	0	0	1-6
main.ts	0	0	0	0	1-17
src/prisma	0	100	0	0	
prisma.module.ts	0	100	100	0	1-8
prisma.service.ts	0	100	0	0	1-16
src/security	100	100	100	100	
security.service.ts	100	100	100	100	
src/tasks	0	0	0	0	
tasks.controller.ts	0	0	0	0	1-49
tasks.module.ts	0	100	100	0	1-11
tasks.service.ts	0	0	0	0	1-64
src/tasks/dto	0	100	100	0	
create-task.dto.ts	0	100	100	0	1-25
update-task.dto.ts	0	100	100	0	1-4
src/tasks/entities	0	100	100	0	
task.entity.ts	0	100	100	0	1
src/users	0	0	0	0	
users.controller.ts	0	0	0	0	1-40
users.module.ts	0	100	100	0	1-11
users.service.ts	0	0	0	0	1-32
src/users/dto	0	100	100	0	
create-user.dto.ts	0	100	100	0	1
update-user.dto.ts	0	100	100	0	1-4
src/users/entities	0	100	100	0	
user.entity.ts	0	100	100	0	1

```

Test Suites: 1 passed, 1 total
Tests: 4 passed, 4 total
Snapshots: 0 total
Time: 2.047 s
Ran all test suites matching src/security/security.service.spec.ts.

```

El archivo security.service.ts obtuvo un coverage del 100% en Statements, Branches, Functions y Lines, garantizando que todas las rutas de ejecución de las funcionalidades RF_9, RF_10 y RF_16 fueron verificadas mediante pruebas unitarias.

Conclusión

Las pruebas ejecutadas validan funcionalidades esenciales del sistema relacionadas con seguridad y privacidad. Los resultados obtenidos muestran que las reglas de negocio implementadas cumplen con los requerimientos RF_9, RF_10 y RF_16 y que todas las rutas de ejecución del módulo de seguridad fueron cubiertas por las pruebas unitarias.

1.5 Autenticación de usuarios (Registro y Login)

Esta funcionalidad permite a los usuarios registrarse en la plataforma e iniciar sesión. Incluye validaciones de seguridad como contraseña con mínimo 8 caracteres que incluya al menos una letra, un número y un símbolo, verificación de email único, y generación de tokens JWT para sesiones.

- 1. Registro de usuario:** Permite a nuevos usuarios crear una cuenta en la plataforma. Debe validar que el email no exista previamente, hashear la contraseña por seguridad, y asignar un rol por defecto.

Casos límite:

- Email duplicado: Intentar registrar con un email que ya existe en la base de datos
- Contraseña en texto plano: Verificar que la contraseña original NO se guarda directamente
- Rol no especificado: Usuario no envía rolName → se asigna "USER" por defecto
- Rol inválido: Usuario envía un rol que no existe en la BD

```

87 describe('register', () => {
88   it('should throw ConflictException if email already exists', async () => {
89     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(mockUser);
90
91     await expect(service.register(mockRegisterDto)).rejects.toThrow(
92       ConflictException,
93     );
94   });
95
96   it('should hash the password before saving', async () => {
97     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(null);
98     jest.spyOn(prismaService.role, 'findUnique').mockResolvedValue(mockRole);
99     jest.spyOn(prismaService.user, 'create').mockResolvedValue(mockUser);
100     (bcrypt.hash as jest.Mock).mockResolvedValue('hashed_password');
101
102     await service.register(mockRegisterDto);
103
104     expect(bcrypt.hash).toHaveBeenCalledWith('MiPassword123!', 10);
105   });
106
107   it('should assign USER role by default', async () => {
108     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(null);
109     jest.spyOn(prismaService.role, 'findUnique').mockResolvedValue(mockRole);
110     jest.spyOn(prismaService.user, 'create').mockResolvedValue(mockUser);
111
112     await service.register(mockRegisterDto);
113
114     expect(prismaService.role.findUnique).toHaveBeenCalledWith({
115       where: { roleName: 'USER' },
116     });
117   });
118
119   it('should return user data and token on success', async () => {
120     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(null);
121     jest.spyOn(prismaService.role, 'findUnique').mockResolvedValue(mockRole);
122     jest.spyOn(prismaService.user, 'create').mockResolvedValue(mockUser);
123     (bcrypt.hash as jest.Mock).mockResolvedValue('hashed_password');
124
125     const result = await service.register(mockRegisterDto);
126
127     expect(result).toEqual({
128       message: 'Usuario registrado exitosamente',
129       user: {
130         id: mockUser.id,
131         name: mockUser.name,
132         email: mockUser.email,
133         role: mockRole.roleName,
134       },
135       token: mockToken,
136     });
137   });
138 });

```

2. Login de usuario: Autentica usuarios existentes. Valida credenciales y genera token JWT.

Casos límite:

- Usuario no existe: Intentar iniciar sesión con un email que no está registrado.
- Contraseña incorrecta: Email existe pero la contraseña enviada no coincide.
- Credenciales correctas: Email y contraseña válidos → retorna usuario y token.

```

140 describe('login', () => {
141   it('should throw UnauthorizedException if user does not exist', async () => {
142     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(null);
143
144     await expect(service.login(mockLoginDto)).rejects.toThrow(
145       UnauthorizedException,
146     );
147   });
148
149   it('should throw UnauthorizedException if password is incorrect', async () => {
150     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(mockUser as any);
151     (bcrypt.compare as jest.Mock).mockResolvedValue(false);
152
153     await expect(service.login(mockLoginDto)).rejects.toThrow(
154       UnauthorizedException,
155     );
156   });
157
158   it('should return user data and token on successful login', async () => {
159     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue({
160       ...mockUser,
161       role: mockRole,
162     } as any);
163     (bcrypt.compare as jest.Mock).mockResolvedValue(true);
164
165     const result = await service.login(mockLoginDto);
166
167     expect(result).toEqual({
168       message: 'Inicio de sesión exitoso',
169       user: {
170         id: mockUser.id,
171         name: mockUser.name,
172         email: mockUser.email,
173         role: mockRole.roleName,
174       },
175       token: mockToken,
176     });
177   });
178 });

```

3. Cambio de contraseña: Permite a usuarios autenticados cambiar su contraseña. Validar contraseña actual y reglas de seguridad.

Casos límite:

- Nueva contraseña igual a actual: Intentar cambiar por la misma contraseña que ya tenía
- Nueva contraseña se hashea: Verificar que la nueva contraseña se guarda hasheada
- Cambio exitoso: Contraseña actual correcta y nueva válida.

```

179 describe('changePassword', () => {
180   const mockChangePasswordDto = {
181     currentPassword: 'MiPassword123!',
182     newPassword: 'NewPassword456!',
183   };
184
185
186   it('should throw BadRequestException if new password equals current password', async () => {
187     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(mockUser);
188     (bcrypt.compare as jest.Mock).mockResolvedValue(true);
189
190     await expect(
191       service.changePassword(1, {
192         currentPassword: 'MiPassword123!',
193         newPassword: 'MiPassword123!',
194       }),
195     ).rejects.toThrow(BadRequestException);
196   });
197
198   it('should hash the new password and update it', async () => {
199     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(mockUser);
200     (bcrypt.compare as jest.Mock).mockResolvedValue(true);
201     (bcrypt.hash as jest.Mock).mockResolvedValue('new_hashed_password');
202     const updateSpy = jest.spyOn(prismaService.user, 'update').mockResolvedValue(mockUser);
203
204     await service.changePassword(1, mockChangePasswordDto);
205
206     expect(bcrypt.hash).toHaveBeenCalledWith('NewPassword456!', 10);
207     expect(updateSpy).toHaveBeenCalledWith({
208       where: { id: 1 },
209       data: { password: 'new_hashed_password' },
210     });
211   });
212
213   it('should return success message on successful password change', async () => {
214     jest.spyOn(prismaService.user, 'findUnique').mockResolvedValue(mockUser);
215     (bcrypt.compare as jest.Mock).mockResolvedValue(true);
216     (bcrypt.hash as jest.Mock).mockResolvedValue('new_hashed_password');
217     jest.spyOn(prismaService.user, 'update').mockResolvedValue(mockUser);
218
219     const result = await service.changePassword(1, mockChangePasswordDto);
220
221     expect(result).toEqual({
222       message: 'Contraseña actualizada exitosamente',
223     });
224   });
225 });
226 });

```

Evidencia de ejecución y coverage

```
C:\Users\Julian B\Documents\project-backend>npm test -- src/auth/auth.service.spec.ts

> freetime-backend@0.0.1 test
> jest src/auth/auth.service.spec.ts

PASS src/auth/auth.service.spec.ts
  AuthService
    register
      ✓ should throw ConflictException if email already exists (12 ms)
      ✓ should hash the password before saving (2 ms)
      ✓ should assign USER role by default (1 ms)
      ✓ should return user data and token on success (1 ms)
    login
      ✓ should throw UnauthorizedException if user does not exist (1 ms)
      ✓ should throw UnauthorizedException if password is incorrect (1 ms)
      ✓ should return user data and token on successful login (1 ms)
    changePassword
      ✓ should throw BadRequestException if new password equals current password (2 ms)
      ✓ should hash the new password and update it (1 ms)
      ✓ should return success message on successful password change (1 ms)

Test Suites: 1 passed, 1 total
Tests: 10 passed, 10 total
Snapshots: 0 total
Time: 0.77 s, estimated 1 s
Ran all test suites matching src/auth/auth.service.spec.ts.

C:\Users\Julian B\Documents\project-backend>
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	19.35	22.22	11.62	20.37	
src	0	0	0	0	
app.controller.ts	0	0	0	0	1-10
app.module.ts	0	100	100	0	1-14
app.service.ts	0	100	0	0	1-6
main.ts	0	0	0	0	1-17
src/auth	58.33	45.45	55.55	60.6	
auth.controller.ts	0	0	0	0	1-27
auth.module.ts	0	0	100	0	1-21
auth.service.ts	93.33	76.92	100	93.02	36,109,115
src/auth/dto	0	100	100	0	
change-password.dto.ts	0	100	100	0	1-12
login.dto.ts	0	100	100	0	1-8
register.dto.ts	0	100	100	0	1-25
src/auth/guards	0	0	0	0	
jwt-auth.guard.ts	0	0	0	0	1-14
src/auth/strategies	0	0	0	0	
jwt.strategy.ts	0	0	0	0	1-16
src/prisma	42.85	100	0	40	
prisma.module.ts	0	100	100	0	1-8
prisma.service.ts	66.66	100	0	57.14	11-19
src/tasks	0	0	0	0	
tasks.controller.ts	0	0	0	0	1-32
tasks.module.ts	0	100	100	0	1-13
tasks.service.ts	0	0	0	0	1-34
src/tasks/dto	0	100	100	0	
create-task.dto.ts	0	100	100	0	1-33
update-task.dto.ts	0	100	100	0	1-4
src/tasks/entities	0	100	100	0	
task.entity.ts	0	100	100	0	1
src/users	0	0	0	0	
users.controller.ts	0	0	0	0	1-32
users.module.ts	0	100	100	0	1-11
users.service.ts	0	0	0	0	1-33
src/users/dto	0	100	100	0	
create-user.dto.ts	0	100	100	0	1-26
update-user.dto.ts	0	100	100	0	1-4
src/users/entities	0	100	100	0	
user.entity.ts	0	100	100	0	1
Test Suites: 1 passed, 1 total					
Tests: 10 passed, 10 total					
Snapshots: 0 total					
Time: 2.562 s					
Ran all test suites matching src/auth/auth.service.spec.ts.					
C:\Users\Julian B\Documents\project-backend>					